Full-batch algorithms for finite sum problems

Implementation of sequential and parallel version using CUDA in python

Marco Trambusti - 7135350

A.A. 2025/2026

Introduction

Purpose of the Assignement

- Implement Logistic Regression with L2 regularization and both Gradient Descent with Armijo Line Search and Conjugate Gradient with Wolfe Line Search algorithms in Python in both sequential and parallelized versions using Numba CUDA.

- Dataset: Binary classification datasets from 768×8 to 11055×68 (rows x features)

- Performance comparison between sequential and parallel version using speedup and efficiency .

Finite-Sum Optimization Problems

- Common in machine learning:
  - Many loss functions are expressed as a sum of independent terms

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x)$$

- Logistic Regression with L2 regularization

$$\min_{w \in \mathbb{R}^p} L(w) + \lambda \Omega(w), L(w) = \sum_{i=1}^{n} \log\left(1 + \exp\left(-y^i w^T x^i\right)\right), \Omega(w) = \frac{1}{2} \sum_{j=1}^{d} w_j^2$$

- **Characteristics:**
  - Unconstrained nonlinear problem
  - Solved using:
    - **Gradient Descent (GD)** with Armijo line search
    - **Conjugate Gradient (CG)** with Wolfe conditions
- **Parallelization:**
  - Loss and gradient computation is naturally parallelizable

Gradient Descent (GD) with Armijo

- **Update rule**: $w_{k+1} = w_k - \alpha_k \nabla f(w_k)$
- **Armijo condition**: $f\left(w_k - \alpha \nabla f(w_k)\right) \leq f(w_k) - \gamma \alpha \parallel \nabla f(w_k) \parallel^2$
- **Key points**:
  - Simple to implement
  - Guarantees convergence
  - Sensitive to step size selection

Algorithm 1 Gradient Descent with Armijo Line Search

$x \leftarrow x_0 \in \mathbb{R}^n$; $k \leftarrow 0$
while $\|\nabla f(x^k)\| > \varepsilon$ **do**
  $d_k \leftarrow -\nabla f(x^k)$
  Compute α_k using Armijo
  $x^{k+1} \leftarrow x^k - \alpha_k \nabla f(x^k)$
  $k \leftarrow k + 1$
end while

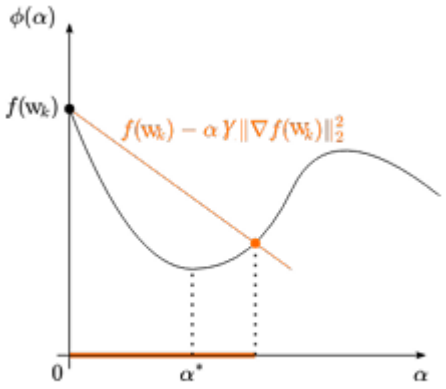


Figure 1. Armijo line search

Conjugate Gradient (CG) with Wolfe

- **Search direction**: $d_{k+1} = -\nabla f(w_{k+1}) + \beta_k d_k, \beta_k = \frac{\|\nabla f(w_{k+1})\|^2}{\|\nabla f(w_k)\|^2}$
- **Wolfe conditions**:

$$f(w_k + \alpha d_k) \leq f(w_k) + \gamma \alpha \nabla f(w_k)^T d_k,$$
$$|\nabla f(w_k + \alpha_k d_k)^T d_k| \leq \sigma |\nabla f(w_k)^T d_k|$$

- **Key points**:
  - Faster convergence than GD
  - More complex line search

Algorithm 2 Conjugate Gradient with Wolfe Line Search

1: $x^0 \in \mathbb{R}^n$, $d_0 \leftarrow -g_0$
2: **while** $\|g_k\| > \varepsilon$ **do**
3: Compute α_k using Wolfe
4: $x^{k+1} \leftarrow x^k + \alpha_k d_k$
5: $g_{k+1} \leftarrow \nabla f(x^{k+1})$
6: $\beta_{k+1} \leftarrow \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$
7: $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$
8: **end while**

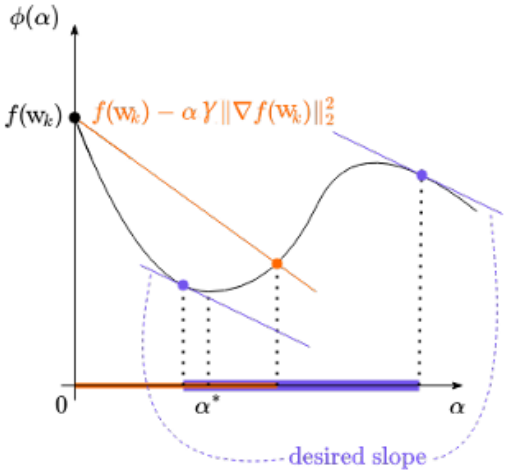


Figure 2. Wolfe conditions

Implementation in Python

- Developed in **Python** using **NumPy** for efficient linear algebra
- **Manual summations** instead of **numpy.sum**
  - Avoids automatic parallelization by libraries like **OpenBLAS**
  - Ensures full control over computational flow
- **Regularization parameter (λ)** optimized via **cross-validation**
  - Used LogisticRegression from scikit-learn
  - Optimal λ values stored in cv_results.csv for reuse

Data Representation and Key Parameters

- **Data loading**:
  - Files from Datasets folder using **load_svmlight_file**
  - X: dense feature matrix; y: binary labels {−1, 1}
  - Normalization applied if needed
  - Converted to float32 for GPU efficiency

- **Parameters in main.py**:
  - **λ**: via cross-validation
  - **Tolerance**: 10^{-4} (stopping criterion)
  - **Armijo**: $\gamma = 10^{-4}$, $\delta = 0.5$
  - **Wolfe**: $\gamma = 10^{-4}$, $\delta = 0.8$, $\sigma = 0.2$

Sequential Implementation

Overview

- Implemented in l2.py as LogisticRegressionL2
- Performs **logistic regression** with **L2 regularization**
- Uses **explicit NumPy operations** (no automatic parallelization)
- Monitors execution time for loss and gradient computations (self.total_time)

Loss Computation

Objective function: $L(w) = \sum_i \log\left(1 + e - y_i x_i^T w\right) + 2\lambda \parallel w \parallel^2$

Implemented in compute_loss():

```
def compute_loss(self, X, y, w):
    ...
    logits = y * np.dot(X, w)
    loss_terms = np.logaddexp(0, -logits)
    total_loss = 0.0
    for i in range(len(loss_terms)):
        total_loss += loss_terms[i]

    reg = 0.0
    for wi in w:
        reg += wi * wi
    regularization = (self.lambda_reg / 2) * reg
    loss = total_loss + regularization
    ...
```

Sequential Implementation

Gradient Computation

- Implemented in `compute_loss_gradient()`:

```
def compute_loss_gradient(self, X, y, w):
    ...
    logits = X.dot(w)
    z = -y * logits
    sig = self.sigmoid(z)
    r = -y * sig
    grad = X.T.dot(r) + self.lambda_reg * w
```

- Combines the **data term** and **regularization gradient**

Parallel Implementation

- Develop a **parallel GPU implementation** using **Numba CUDA**
- Parallelize loss & gradient computation

Parallelization Points

- Operations split into **CUDA kernels** exploiting data independence
- Each **thread** processes a sample or a feature
- Partial results combined via **reduction kernels**

- Key kernels:
  - `compute_loss_cuda_kernel_global`
  - `compute_r_kernel`
  - `compute_XTr_kernel`

Loss Kernel

```
@cuda.jit(fastmath=True)
def compute_loss_cuda_kernel_global(X_flat, y, w, loss_terms, n_samples,
    tx = cuda.threadIdx.x
    bx = cuda.blockIdx.x
    bw = cuda.blockDim.x
    grid_size = cuda.gridDim.x

    for i in range(bx, n_samples, grid_size):
        partial_dot = 0.0
        for j in range(tx, n_features, bw):
            index = i * n_features + j
            partial_dot += X_flat[index] * w[j]

        dot = block_reduce_sum(partial_dot)

        if tx == 0:
            y_i = y[i]
            z = -y_i * dot
            loss = z + math.log1p(math.exp(-z)) if z > 0 \
                    else math.log1p(math.exp(z))
            loss_terms[i] = loss
```

Computes logistic loss for each sample

- Uses **fastmath=True** for optimized math ops (exp, log1p)
- Small precision trade-off for speed
- Aggregation through block_reduce_sum for thread-level results

Reduction Kernels

```python
@cuda.jit(device=True)
def warp_reduce_sum(val):
    mask = cuda.warpsize // 2
    while mask > 0:
        val += cuda.shfl_xor_sync(0xFFFFFFFF, val, mask)
        mask //= 2
    return val
```

```python
@cuda.jit(device=True)
def block_reduce_sum(val):
    warp_size = cuda.warpsize
    num_warps = cuda.blockDim.x // warp_size
    shared = cuda.shared.array(shape=32, dtype=float64)

    lane = cuda.threadIdx.x % warp_size
    wid = cuda.threadIdx.x // warp_size

    val = warp_reduce_sum(val)

    if lane == 0 and wid < num_warps:
        shared[wid] = val
    cuda.syncthreads()

    val = 0.0
    if wid == 0 and lane < num_warps:
        val = shared[lane]
    val = warp_reduce_sum(val)
    return val
```

Aggregate partial results (e.g., dot products, L2 norm)

- **Warp-level reduction:** `warp_reduce_sum(val)`
- **Block-level reduction:** `block_reduce_sum(val)`
- Use of:
  - **Shared memory**
  - **Thread synchronization**
- Enables efficient hierarchical aggregation on GPU

Gradient Computation

```
@cuda.jit@cuda.jit(fastmath=True)
def compute_r_kernel(X_flat, y, w, r, n_samples, n_features):
    i = cuda.grid(1)
    if i < n_samples:
        dot = 0.0
        base = i * n_features
        for j in range(n_features):
            dot += X_flat[base + j] * w[j]
        z = -y[i] * dot
        r[i] = -y[i] * sigmoid(z)
```

```
@cuda.jit@cuda.jit(fastmath=True)
def compute_XTr_kernel(X_flat, r, grad, n_samples, n_features)
    j = cuda.grid(1)
    if j < n_features:
        acc = 0.0
        for i in range(n_samples):
            acc += X_flat[i * n_features + j] * r[i]
        grad[j] = acc
```

Two main kernels:

1. **`compute_r_kernel`**: Computes intermediate vector $r_i = -y_i \cdot \sigma\left(-y_i x_i^T w\right)$
2. **`compute_XTr_kernel`**: Computes gradient $X^T r$ in parallel
   - Each thread handles one feature

L2 regularization term handled on **CPU** — simpler and avoids data transfer overhead.

Warmup: Kernel Pre-Initialization

```
X_dummy = np.random.randn(N, D).astype(np.float32)
y_dummy = np.random.choice([-1, 1], size=N).astype(np.float32)
w_dummy = np.random.randn(D).astype(np.float64)

compute_loss_cuda_kernel_global...
compute_r_kernel...
compute_XTr_kernel...
```

- Numba compiles kernels **on first call (JIT)** → initial latency
- **Solution:** `warmup_all()` method pre-compiles kernels using dummy data
- Ensures:
  - No delay during actual optimization
  - Early verification of GPU pipeline
  - Memory & type checks before real run

Results and Performance Evaluation

Setup:

- Tests conducted on an Intel Core i7-6700K processor (4 cores and 8 threads), and a NVIDIA GeForce GTX 970 GPU, based on Maxwell architecture, features 1664 CUDA cores, a base clock speed of 1050 MHz, 4 GB of GDDR5 memory with a 256-bit memory interface.
- Data sets of various sizes with a commensurate number of clusters and epochs.
- The results are averaged across five runs.

Results and Performance Evaluation

Duration:
Average execution time of each configuration.

- operations on larger data sets take significantly longer.

Small Dataset (768 × 8)

- GD: 4.91 s → 5.88 s (CUDA 256 threads)
- CG: 2.19 s → 2.81 s (CUDA 256 threads):

Kernel launch and synchronization overhead exceed computation benefits.

Large Dataset (3185 × 122)

- GD: 477.03 s → 128.12 s (CUDA 512 threads, 3.7× faster)
- CG: 336.25 s → 66.56 s (CUDA 512 threads, 5.0× faster)

Parallelization effective: GPU well utilized.

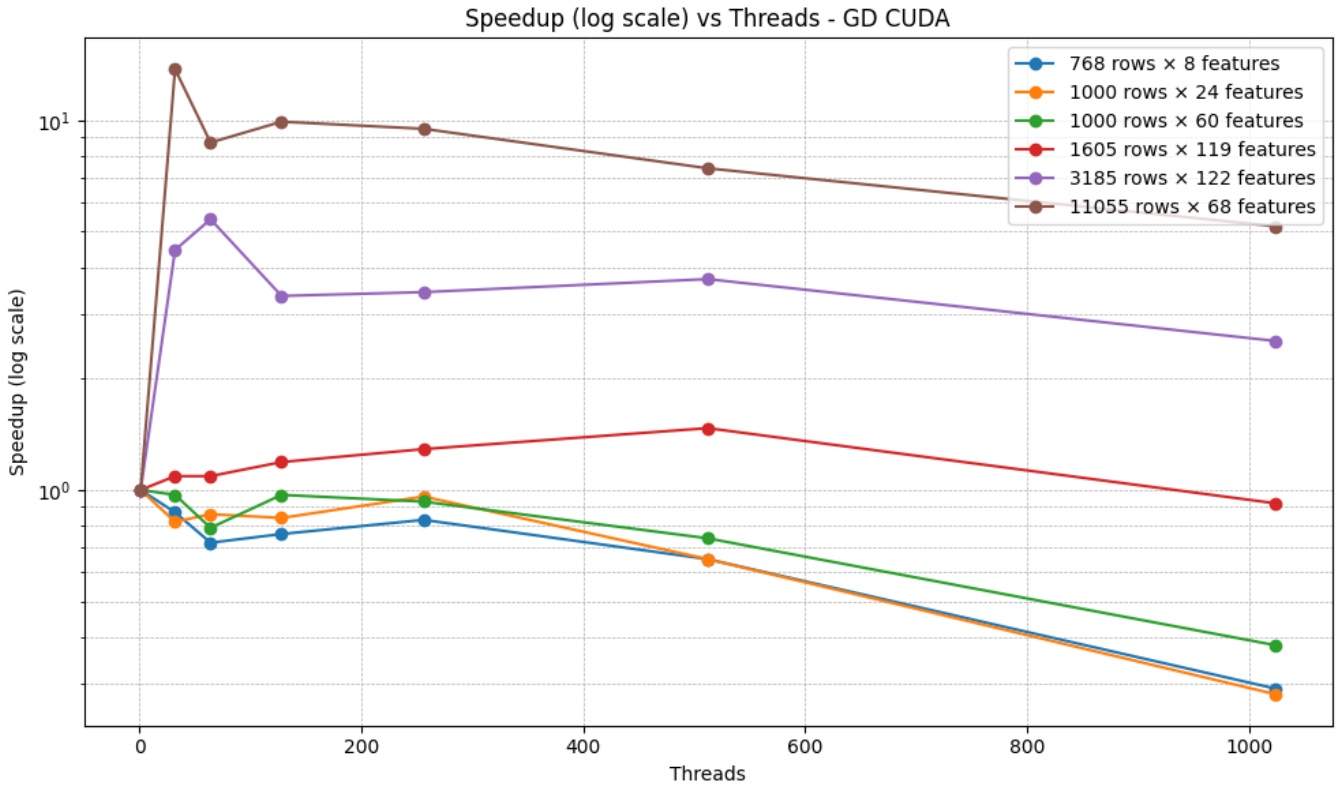
Very Large Dataset (11,055 × 68)

- GD Sequential: 670.30 s → 48.72 s (CUDA 32 threads, 13.8× faster)
- CG Sequential: 380.74 s → 41.98 s (CUDA 32 threads, 9.1× faster)

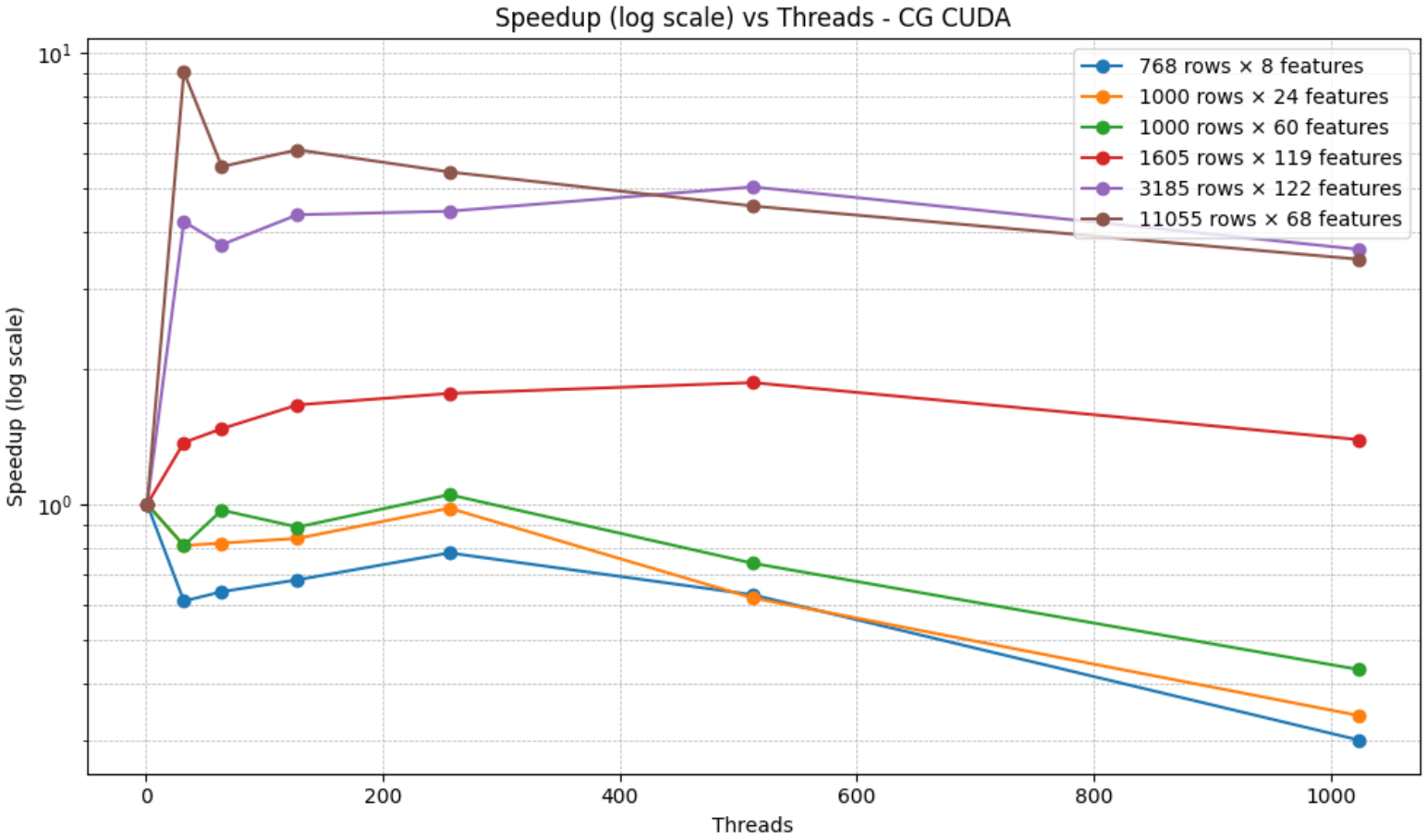
Excellent scalability — parallel execution dominates overhead.

Results and Performance Evaluation

Speedup: Ratio of sequential to parallel execution time.



Speedup (CG)


Speedup (log scale) vs Threads - CG CUDA
Speedup (log scale)
Threads
10^1
10^0
0
200
400
600
800
1000
768 rows × 8 features
1000 rows × 24 features
1000 rows × 60 features
1605 rows × 119 features
3185 rows × 122 features
11055 rows × 68 features

Speedup:

Small Datasets (768×8, 1000×24, 1000×60):

- Modest total speedup (0.7× - 1.2×) due to overhead and kernel complexity.
- Gradient speedup: 0.97× to 1.21×
- Loss speedup: 0.71× to 0.98×

Medium Datasets (1605×119, 3185×122):

- Total speedup:
  - GD CUDA up to 5.40×
  - CG CUDA up to 6.11×
- Gradient speedup: 2.46× to 6.11×
- Loss speedup: 1.34× to 4.60×

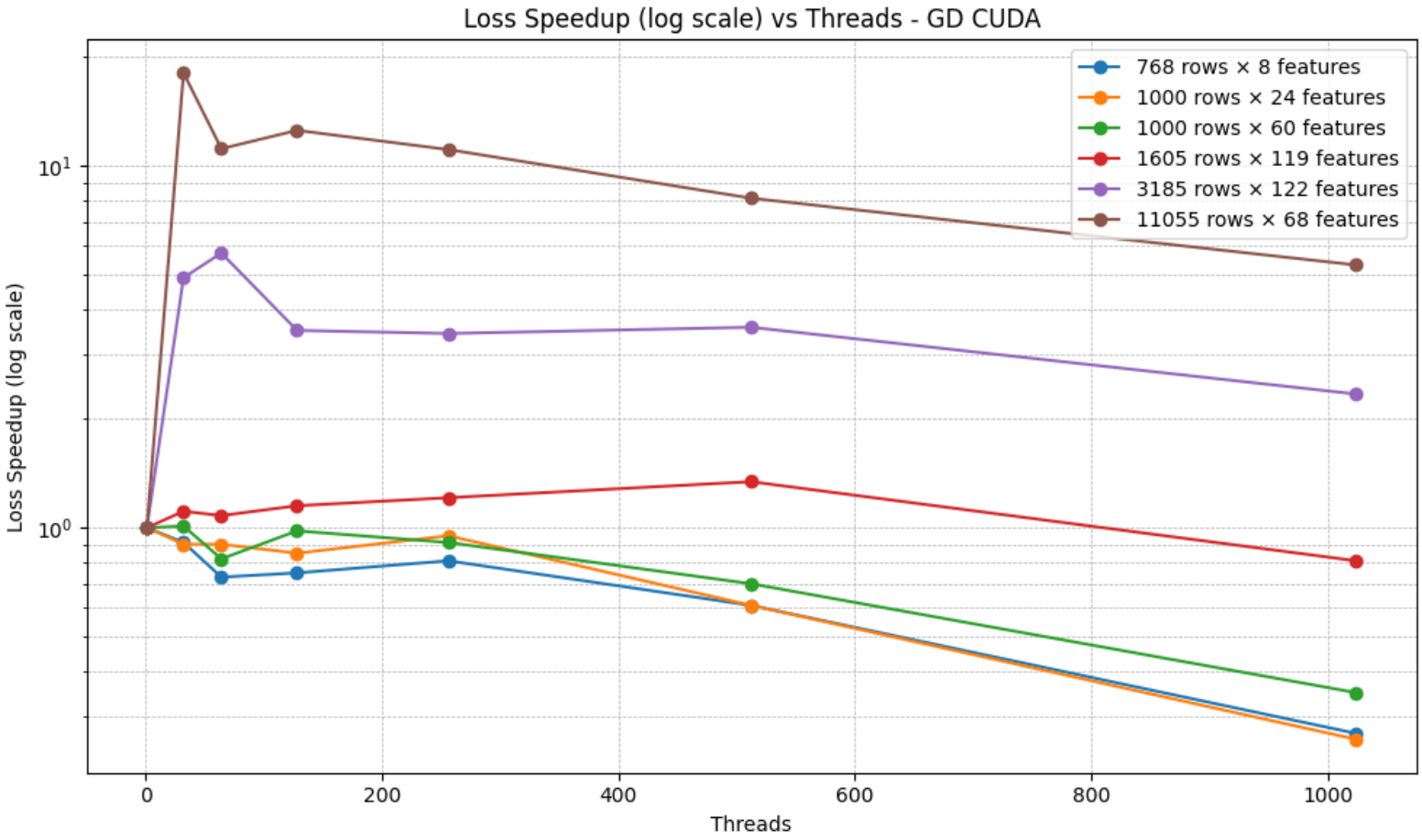
Large Dataset (11,055×68):

- Total speedup:
  - GD CUDA 13.76×
  - CG CUDA 9.07×
- Gradient speedup: 3.99× to 4.44×
- Loss speedup: up to 18.06× (GD CUDA) and 16.04× (CG CUDA)

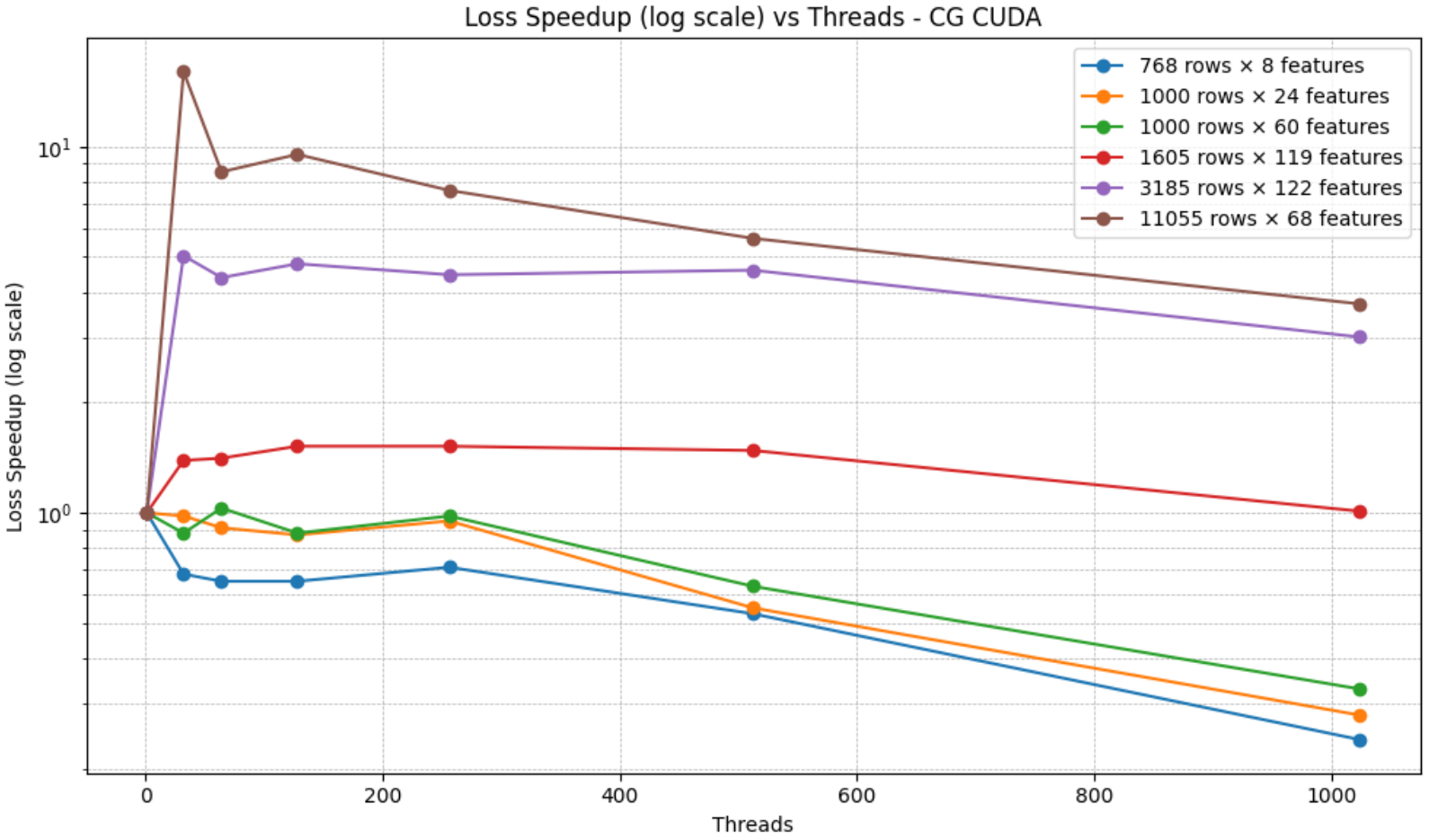
General Observations:

- Loss computation shows the highest speedups due to its parallel nature.
- Gradient computation is more complex, but speedup increases with data size.

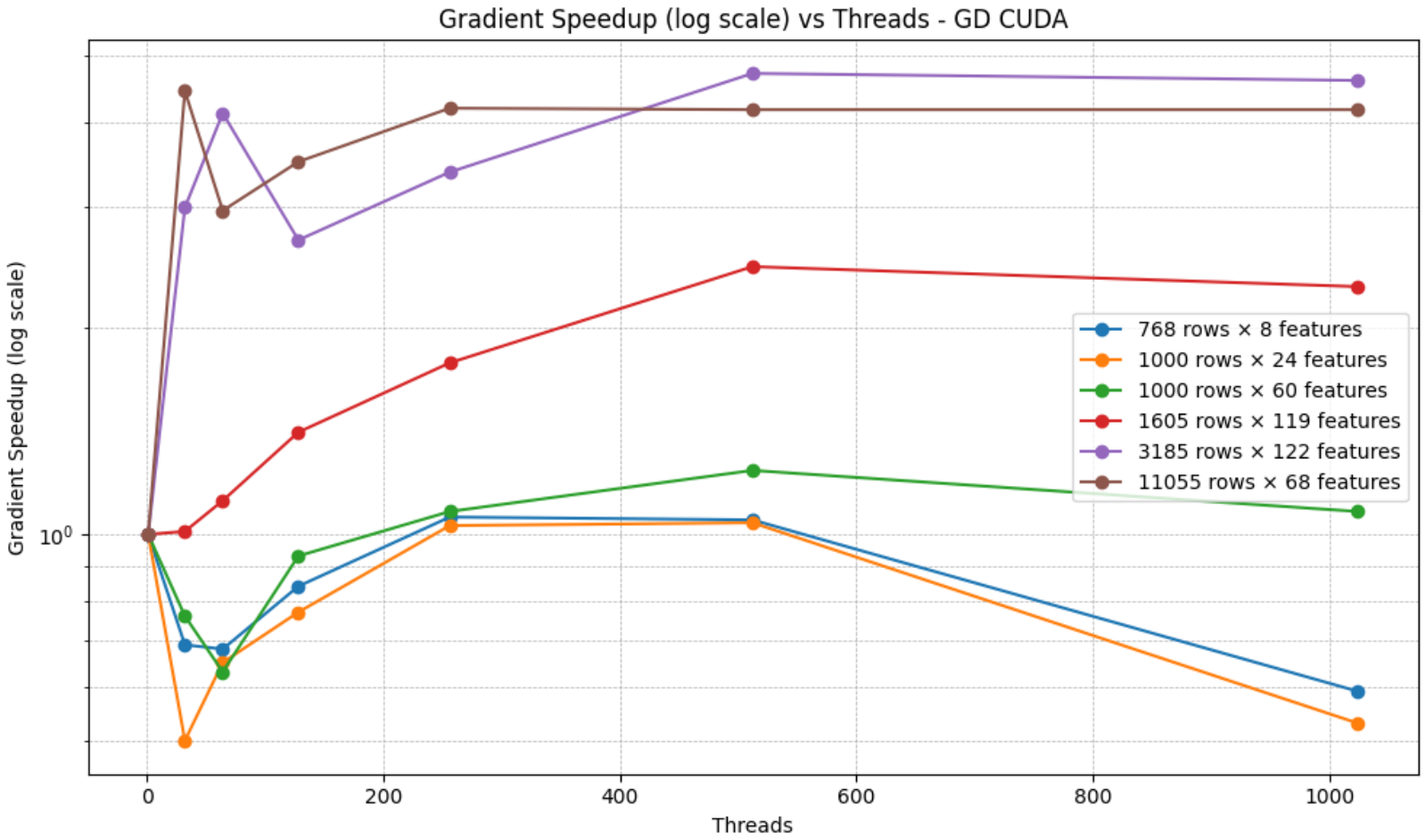
Loss Speedup (GD)



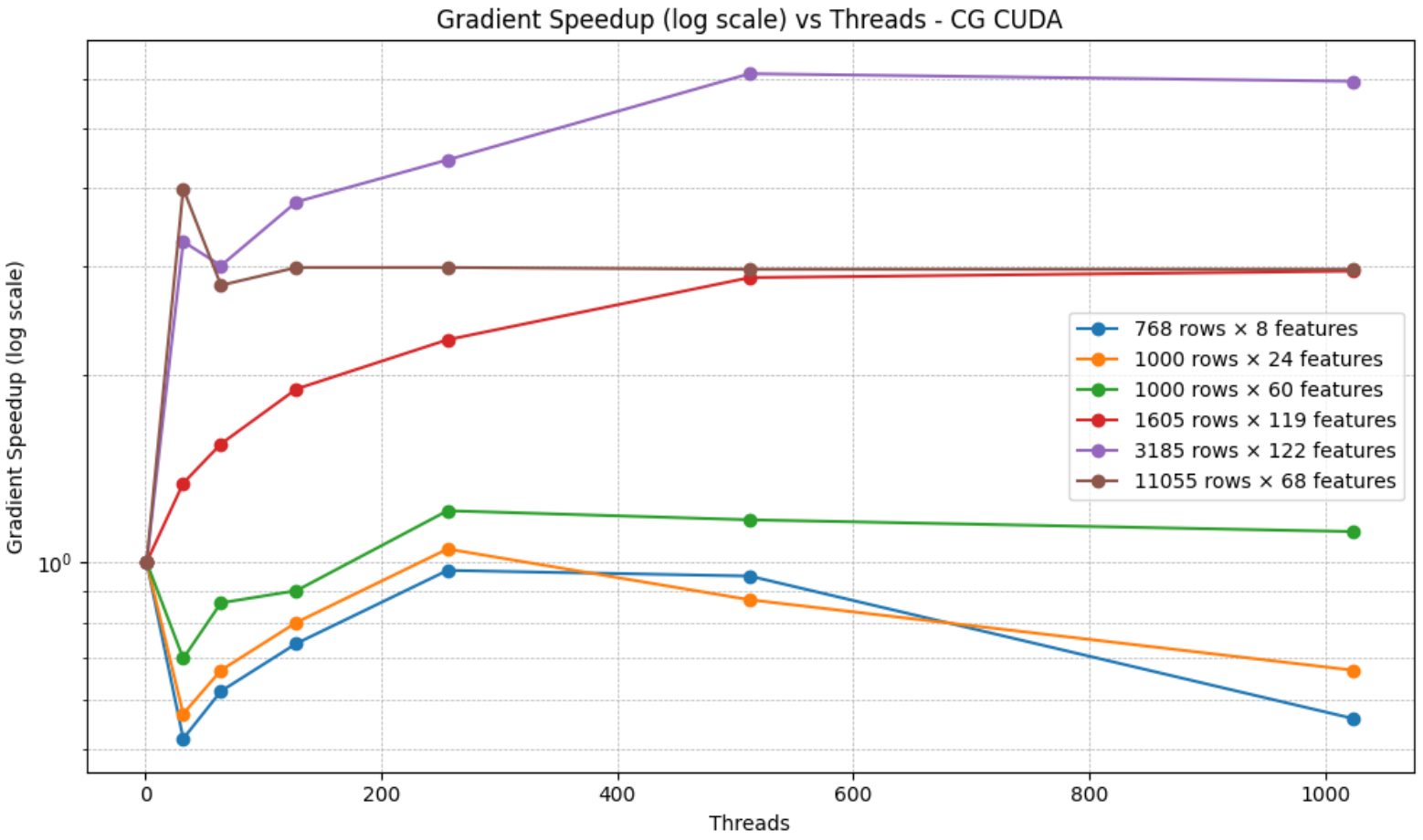
Loss Speedup (CG)



Gradient Speedup (GD)

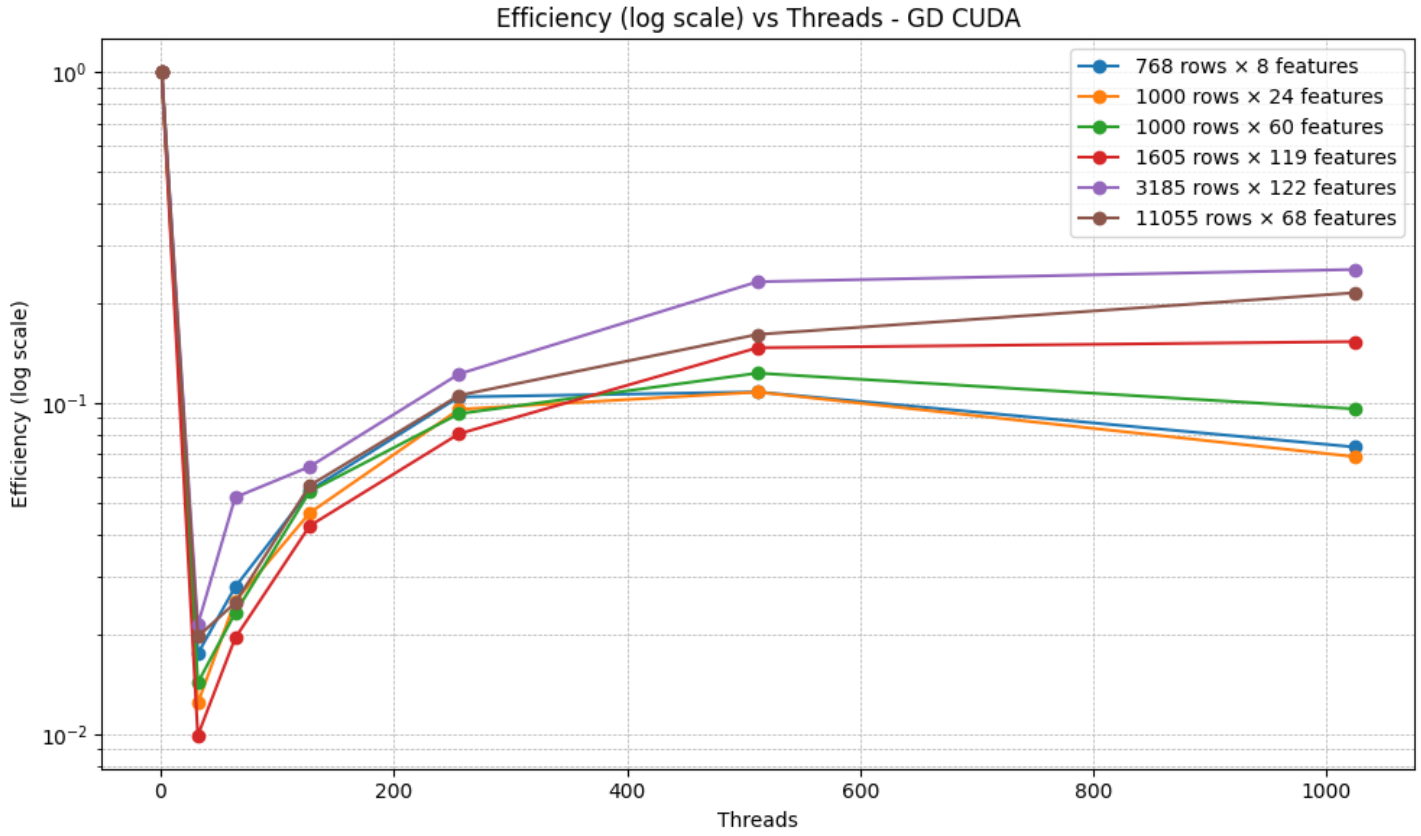


Gradient Speedup (CG)

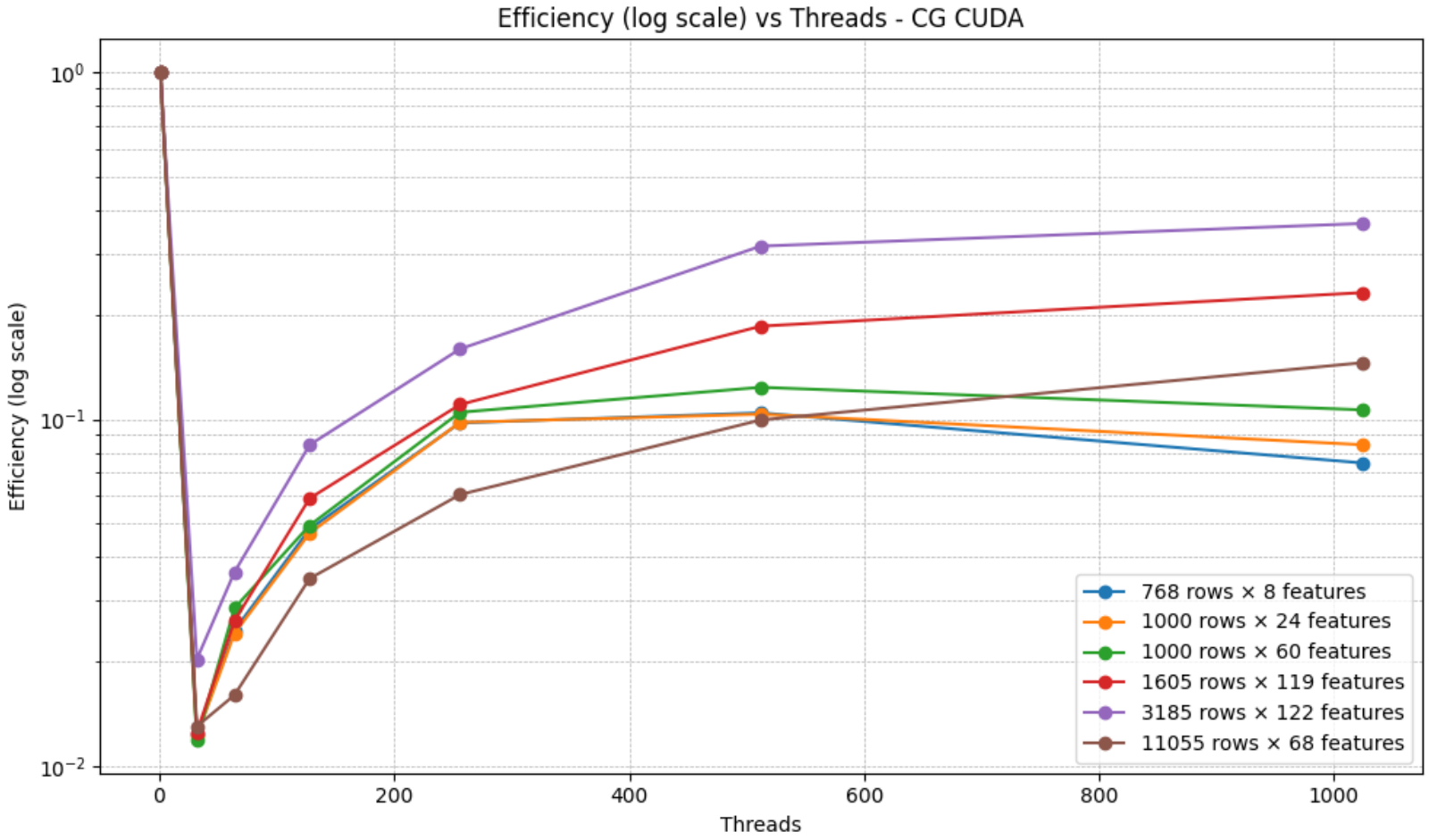


Results and Performance Evaluation

Efficiency: Ratio of Speedup to Number of Threads.



Efficiency (CG)



Efficiency:

Small Datasets (768×8, 1000×24, 1000×60)

Total efficiency: 0.104–0.123 (GD), 0.104–0.123 (CG)
Gradient efficiency: 0.103–0.133 (GD), 0.121–0.166 (CG)
Loss efficiency: 0.069–0.116 (GD), 0.070–0.089 (CG)
→ *Low efficiency due to parallelization overhead.*

Medium Datasets (1605×119, 3185×122)

Total efficiency: 0.147–0.460 (GD), 0.232–0.594 (CG)
Gradient efficiency: 0.246–0.460 (GD), 0.232–0.594 (CG)
Loss efficiency: 0.134–0.301 (GD), 0.169–0.302 (CG)
→ *Higher TPB improves performance, especially for CG CUDA.*

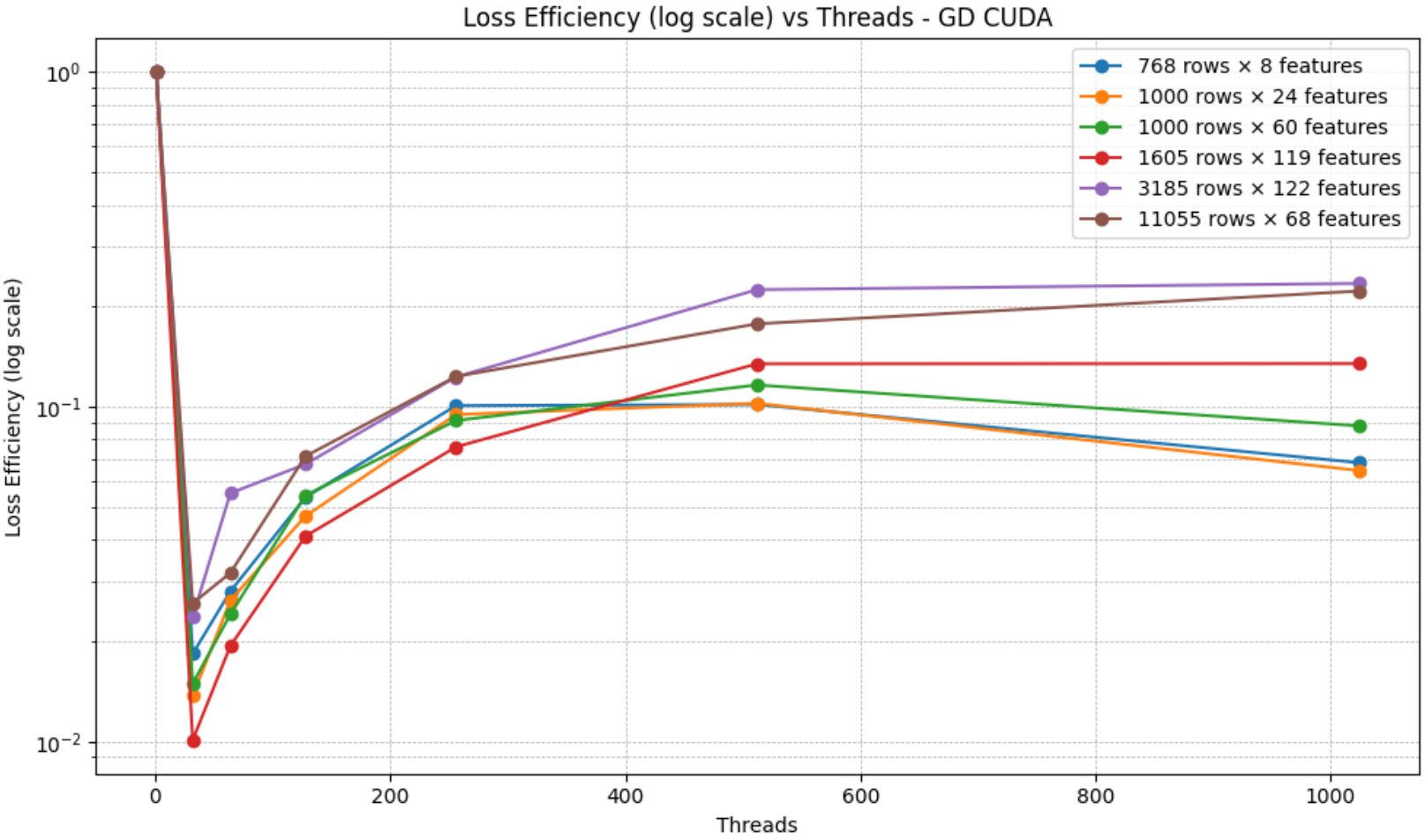
Large Dataset (11,055×68)

Total efficiency: GD 0.2155, CG 0.1455
Gradient efficiency: GD 0.1737, CG 0.1235
Loss efficiency: GD 0.2216, CG 0.1549
→ *Peak efficiency for CG CUDA with TPB = 1024.*

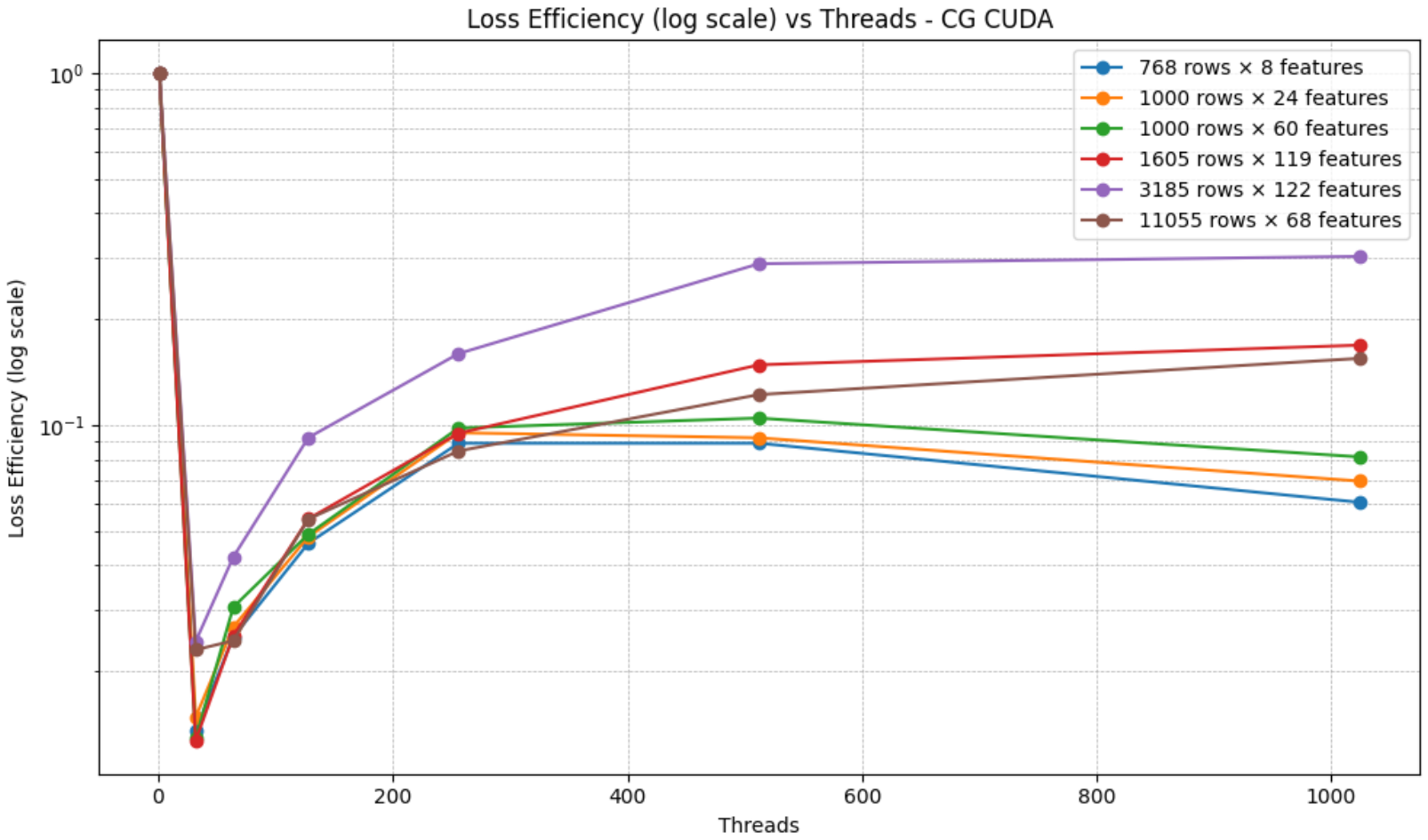
Key Insights:

- Efficiency grows with dataset size and optimal TPB.
- Loss computation benefits more from parallelism than gradient computation.

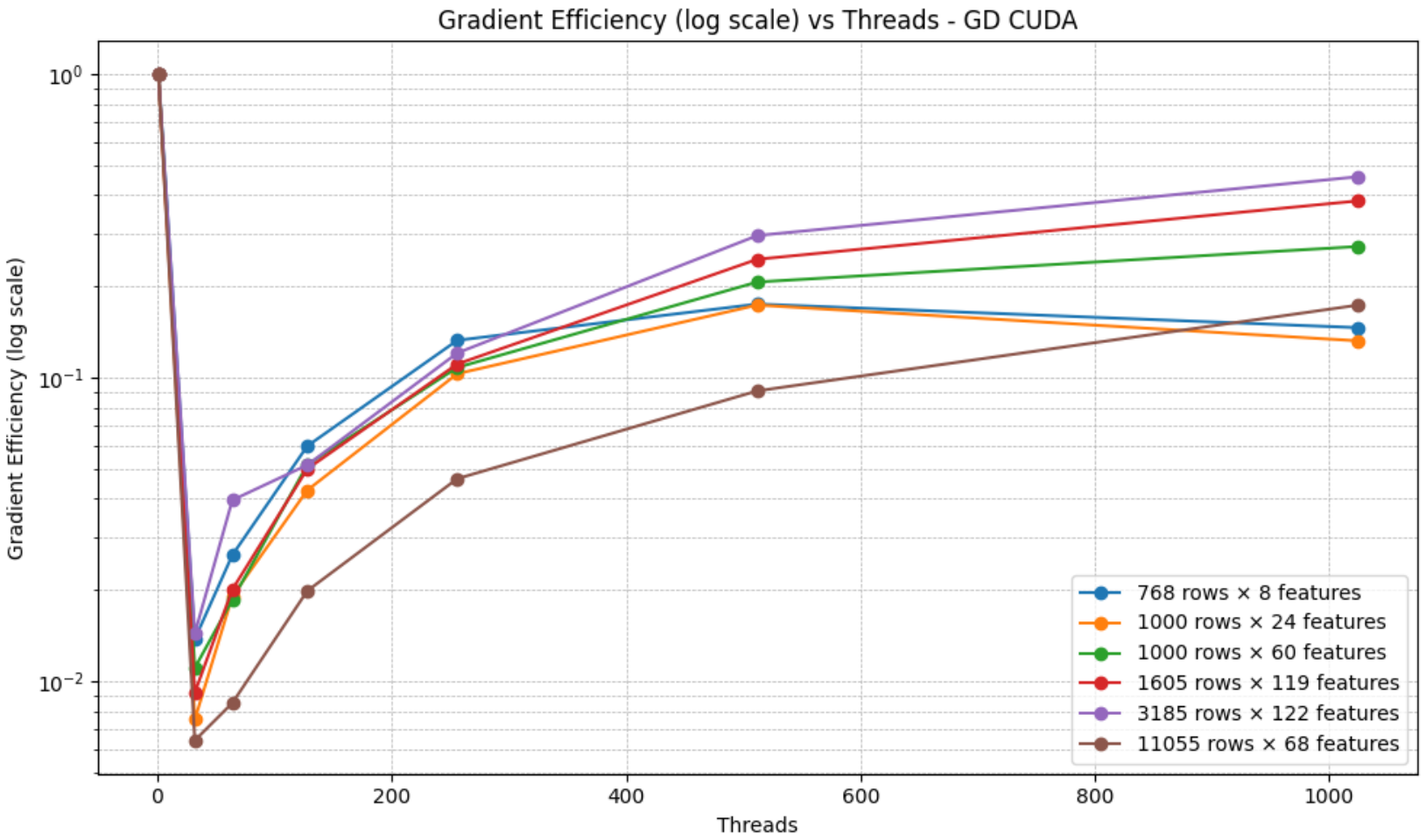
Loss Efficiency (GD)



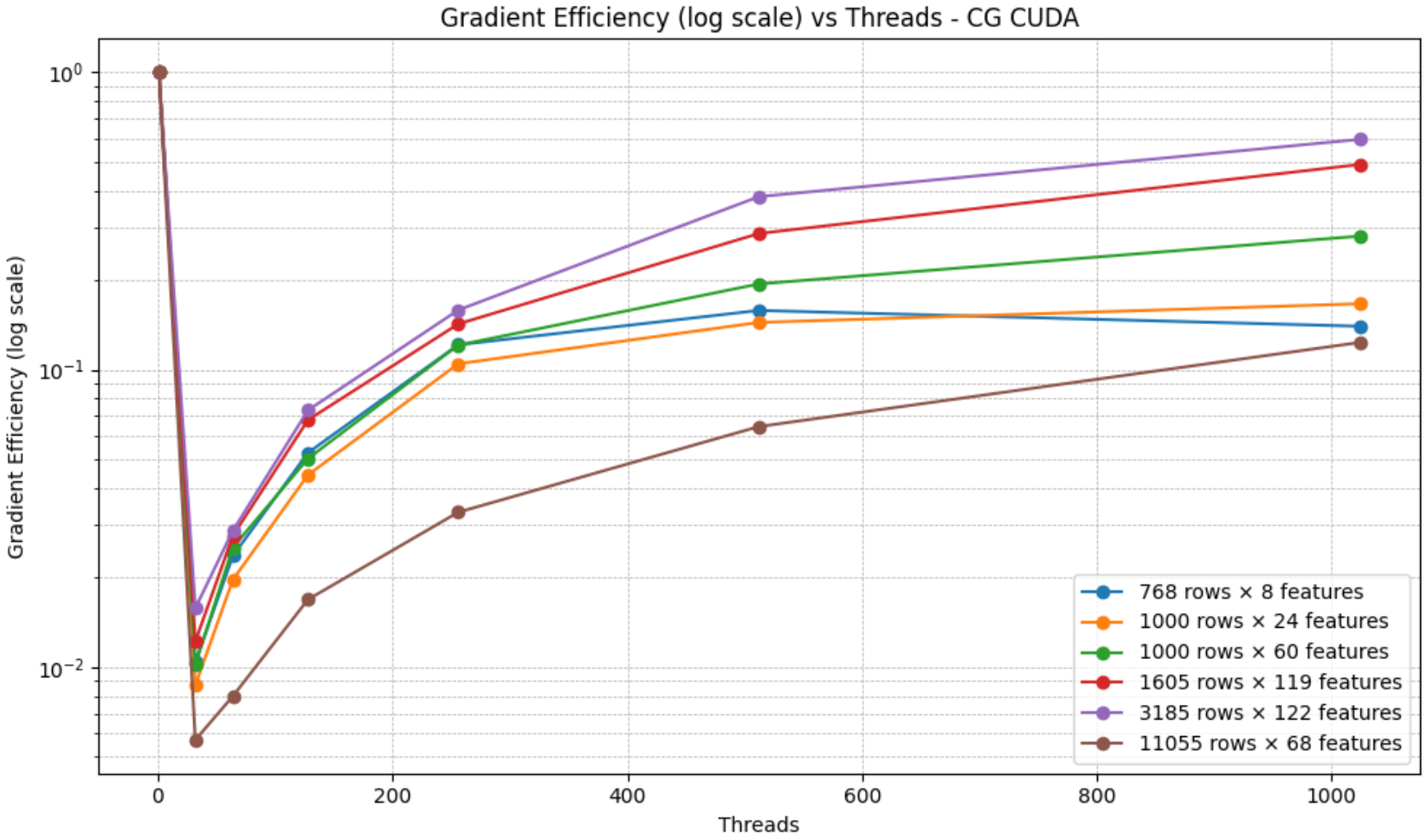
Loss Efficiency (CG)



Gradient Efficiency (GD)



Gradient Efficiency (CG)



Conclusions

General Findings:

- Significant Speedup performance gains for large datasets.
- Efficiency is very low and decreases as the number of threads increases

Performance Characteristics:

- Minimal speedup for smaller datasets due to overhead.
- Loss computation generally benefits more from parallelism.

Key Takeaway:

- CUDA parallelization is effective for large datasets, but efficiency diminishes with excessive threads; a balance is crucial for optimal performance.